Introduction to the Theory of Computation

Lecture 11. Church–Turing Thesis

Neo (Yuan) Li

Oct 2026

Fudan University

yuan_li@fudan.edu.cn

RISC-V RV32I: The Machine Model

RISC-V is an open, modern instruction set architecture.

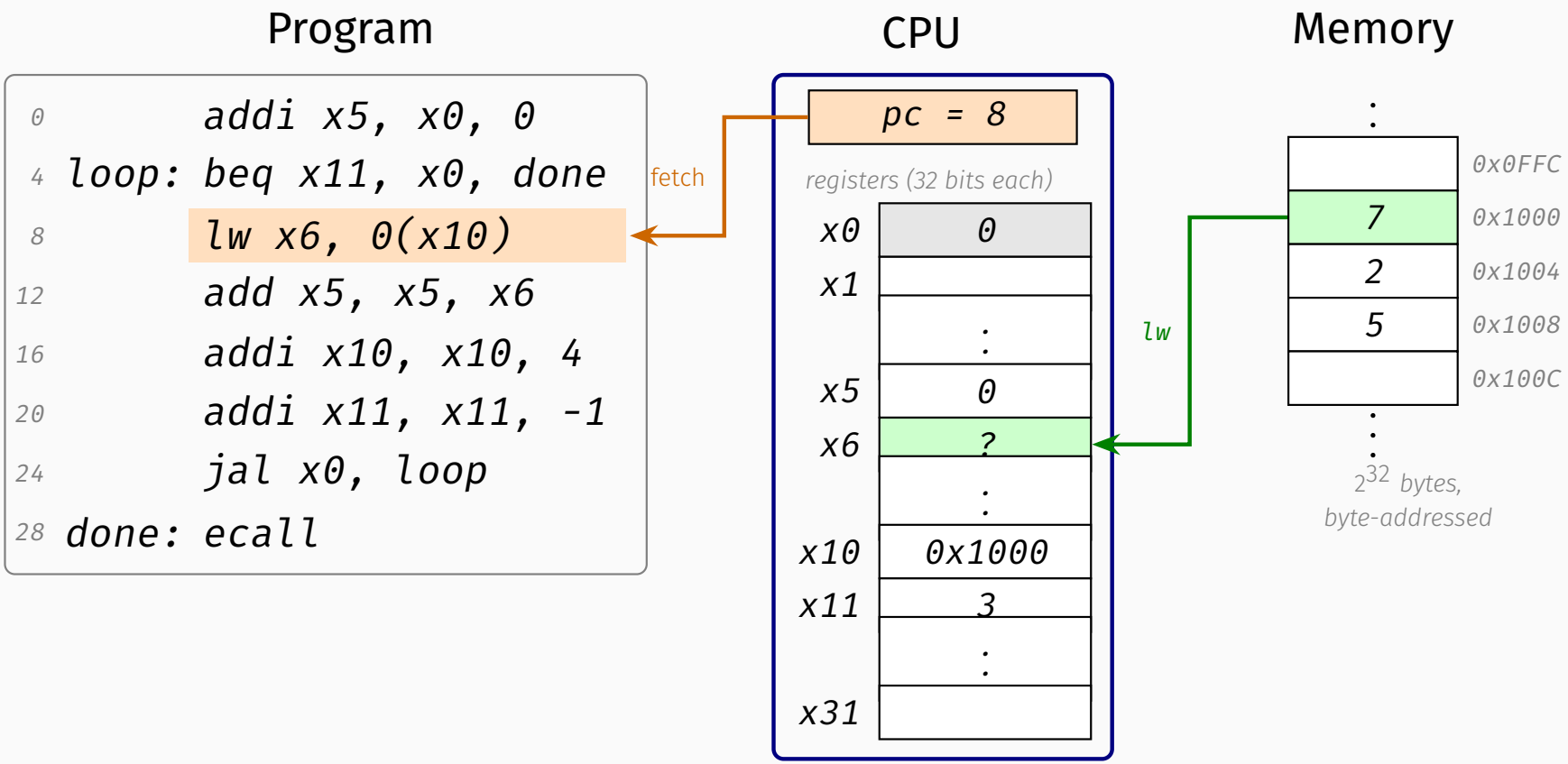
Machine state:

- 32 registers *x0*, ..., *x31*, each 32 bits; *x0* is hardwired to 0.
- A program counter *pc*.
- Byte-addressed memory, addresses are 32 bits.

Design features:

- Load/store architecture: only loads and stores touch memory; arithmetic works on registers only.
- No condition flags, no multiplication/division in the base set.

RISC-V RV32I: What the Machine Looks Like



C code: `while (n) { sum += *p; p++; n--; }` $x10 = p,\ x11 = n,\ x5 = sum$

- `pc` points to the next instruction
- `lw x6, 0(x10)`: $x6 \leftarrow \text{Mem}[0x1000]$

Encoding RV32I on a RAM TM

Fix an RV32I program P. Build a RAM TM M_P:

- Each register *x1*, ..., *x31* gets **its own work tape**, holding 32 bits.
- Memory byte at address a is stored in $A[a]$.
- The *pc* is kept in the **finite control**: one group of states per instruction of P.

Assumption. The word size 32 is a constant, and P is fixed (not modified at run time).

RV32I Instructions by Category (1/2)

Category	Instructions	Effect
Upper immediate	*LUI*, *AUIPC*	$rd \leftarrow imm{<<}12 \ (+ pc)$
ALU, reg–imm	*ADDI SLTI SLTIU*	$rd \leftarrow rs1 \circ imm$
	XORI ORI ANDI	
	SLLI SRLI SRAI	
ALU, reg–reg	*ADD SUB SLT SLTU*	$rd \leftarrow rs1 \circ rs2$
	XOR OR AND	
	SLL SRL SRA	

$\circ \in \{+,\ -,\ <_{\text{signed}},\ <_{\text{unsigned}},\ \oplus,\ \vee,\ \wedge,\ \ll,\ \gg_{\text{logical}},\ \gg_{\text{arith}}\}$

RV32I Instructions by Category (2/2)

Category	Instructions	Effect
Load	*LB LH LW LBU LHU*	*rd* ← Mem[*rs1*+*off*]
Store	*SB SH SW*	Mem[*rs1*+*off*] ← *rs2*
Branch	*BEQ BNE BLT*	if *rs1* ∼ *rs2*:
	BGE BLTU BGEU	*pc* ← *pc*+*off*
Jump	*JAL*	*rd* ← *pc*+4; *pc* ← *pc*+*off*
	JALR	*rd* ← *pc*+4; *pc* ← *rs1*+*off*
System	*FENCE ECALL EBREAK*	ordering / OS call / debug

Total: 40 instructions.

Building Blocks: Word Operations in $O(1)$

On 32-bit words stored on work tapes, a multitape TM can do each of the following in $O(1)$ steps:

- copy a word, or write a constant;
- add, subtract (mod 2^{32});
- bitwise $\wedge$, $\vee$, $\oplus$;
- shift left/right by s bits ($0 \le s < 32$);
- compare (signed or unsigned), sign/zero-extend.

Memory access. Writing a 32-bit address on the address tape takes $O(1)$ steps; one read/write of A costs $O(1)$ by the RAM TM definition.

Each Instruction Costs $O(1)$ (1/2)

ALU (reg–reg, reg–imm). Read *rs1* (and *rs2*) from their tapes, compute one word operation, write the result to *rd*'s tape. $O(1)$

Upper immediate. *imm* is part of the instruction, and *pc* is known to the finite control. Write the constant *imm<<12* (or *pc*+*imm<<12*) to *rd*'s tape. $O(1)$

Load / Store. Compute *rs1*+*off*, write it on the address tape, then do ≤ 4 reads/writes of A (one per byte), plus sign/zero-extension. $O(1)$

Each Instruction Costs $O(1)$ (2/2)

Branch. Compare $rs1$ and $rs2$; depending on the result, move to the state group of instruction $pc{+}off$ or $pc{+}4$. $O(1)$

JAL. Write the constant $pc{+}4$ to rd; switch to the state group of $pc{+}off$. $O(1)$

JALR. Compute $rs1{+}off$ on a scratch tape; scan its 32 bits to select among the finitely many instruction addresses of P. $O(1)$

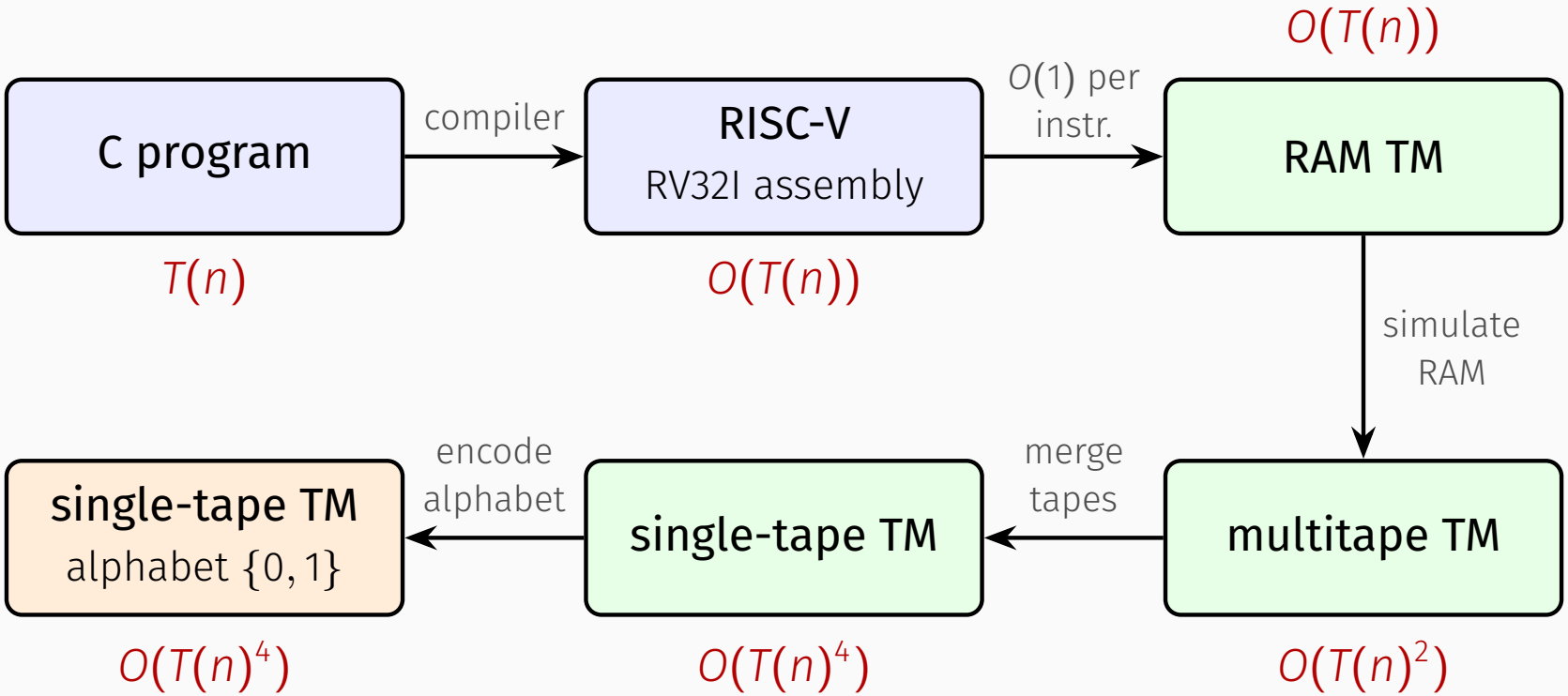
System. *FENCE*: no-op. *ECALL*, *EBREAK*: halt (or output). $O(1)$

Conclusion

Every RV32I instruction can be simulated by a multitape RAM TM in $O(1)$ steps.

Every RV32I program that runs in time $T(n)$ can be simulated by a multitape RAM TM in time $O(T(n))$.

From C to a Turing Machine



Every C program running in time $T(n)$ can be simulated by a single-tape TM over $\{0,1\}$ in time $O(T(n)^4)$ — **polynomial**.

Remark: The Word Size

The $O(1)$ bound used that the word size $w = 32$ is a **constant**.

- Then memory has at most 2^{32} bytes — strictly a finite machine. Fine in practice, but an idealization.
- If we let w grow (e.g. $w = \Theta(\log n)$, so that memory can hold the whole input), each instruction costs $O(w)$, and the simulation costs $O(T(n) \cdot w)$.
- Multiplication (RV32M extension) costs $O(w^2)$ with the grade-school algorithm.

Either way the overhead is **polynomial**.

“$O(T(n))$” — In Which Model?

Community	Default model	One time step	Typical statement
Algorithms	**word RAM**, word size $w = \Theta(\log n)$	one operation on a w-bit word	sorting in $O(n \log n)$; hashing in $O(1)$
Complexity	**multitape TM**	one head move / one gate	P, NP; integer multiplication in $O(n \log n)$

Other Models Equivalent to Turing Machines

Model	Idea
Register Machine	*INC*/*DEC*/*JZ* on counters
λ-calculus / SKI	β-reduction
μ-Recursive Functions	prim. recursion + μ
Post / Markov Systems	string rewriting
Cellular Automata	local rules (Rule 110)
Boolean Circuits	circuit families

Church–Turing Thesis

Church–Turing Thesis. *Every function that is effectively computable is computable by a Turing machine.*

The Strong Church–Turing Thesis

Strong Church–Turing Thesis. *Any reasonable model of computation can be simulated by a Turing machine with at most* ***polynomial*** *overhead.*

? Possibly false: quantum computers

Challenge to the Strong Church–Turing Thesis

Quantum computers.

- **Shor (1994):** factoring in $\mathrm{poly}(n)$ time on a quantum computer.
- Best known classical algorithm (number field sieve): $2^{O(n^{1/3}(\log n)^{2/3})}$ — **sub-exponential**.
- Widely believed: BPP $\subsetneq$ BQP — but unproven.

If true, the Strong CTT **fails** for quantum computers.

The Physical Church–Turing Thesis

Physical Church–Turing Thesis. *Every function that can be computed by a* ***physical system*** *is computable by a Turing machine.*

- A claim about the **laws of physics**, not about algorithms.
- **Quantum computers** do not violate it: they are (possibly) faster, but compute the same functions.
- Proposed counterexamples (“hypercomputation”) rely on idealizations: infinite-precision real numbers, exotic spacetimes, …

Alternative Model 1: Register Machines (Minsky)

A **register machine** has finitely many registers $r_1, r_2, \ldots,$ each holding a natural number (initially 0), and a program consisting of numbered instructions of only two kinds:

Increment: $\ell:\ \mathit{INC}(r);\ \text{goto } \ell'$

- Effect: $r \mathrel{+}= 1$, then jump to instruction ℓ'.

Decrement-and-branch: $\ell:\ \mathit{DEC}(r);\ \text{goto } \ell_1 \text{ else } \ell_2$

- If $r > 0$: $r \mathrel{-}= 1$, then jump to ℓ_1.
- If $r = 0$: leave r unchanged, jump to ℓ_2 instead.

Alternative Model 2: Cellular Automata

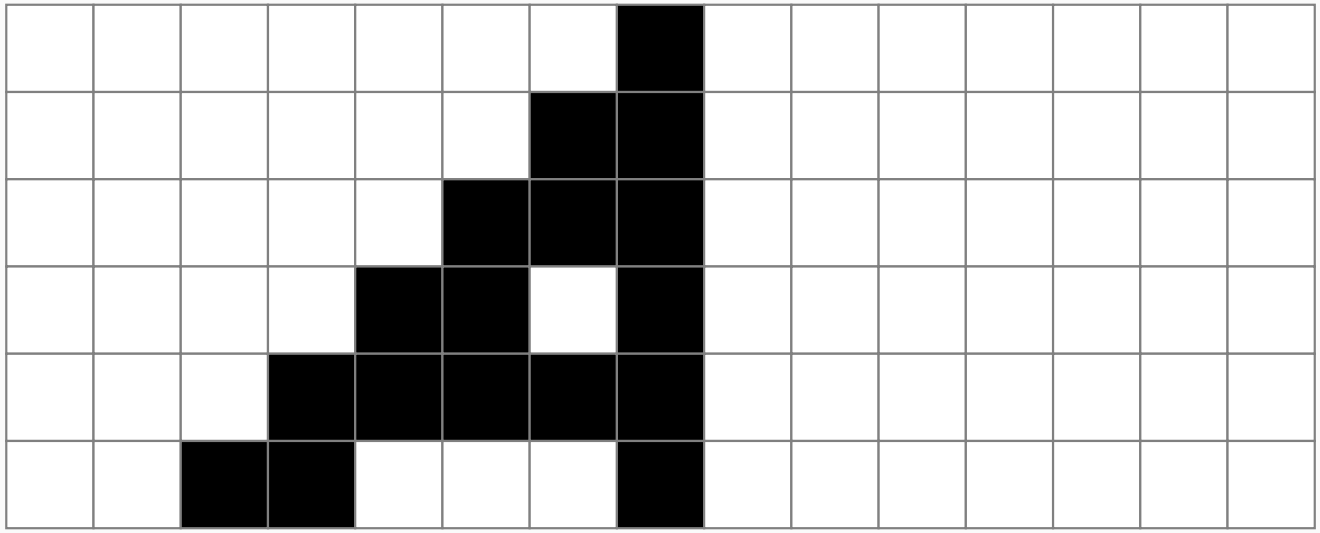


Figure 1: Rule 110: each cell's next state depends only on itself and its two neighbors.

Alternative Model 2: Cellular Automata

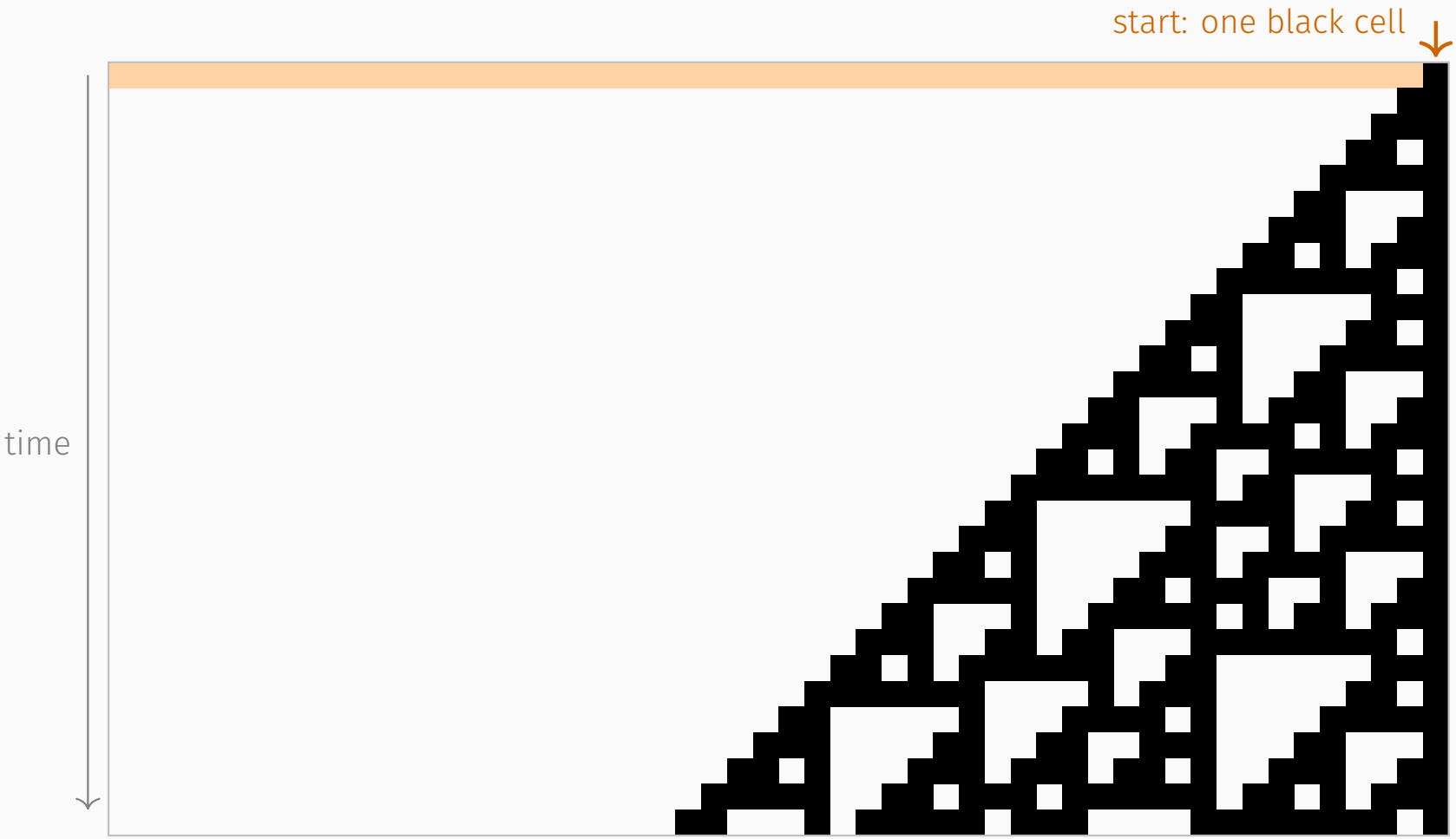
A *cellular automaton* is an infinite row of cells, each in state 0 (white) or 1 (black). At every time step, **every cell updates simultaneously**, using only a fixed rule applied to itself and its two neighbors.

Rule 110 (the entire update rule — one output bit per possible 3-cell neighborhood):

left, self, right	111	110	101	100	011	010	001	000
new state	0	1	1	0	1	1	1	0

Initial condition: all cells 0, except a single cell set to 1 (row 0 below). Each subsequent row is one time step later.

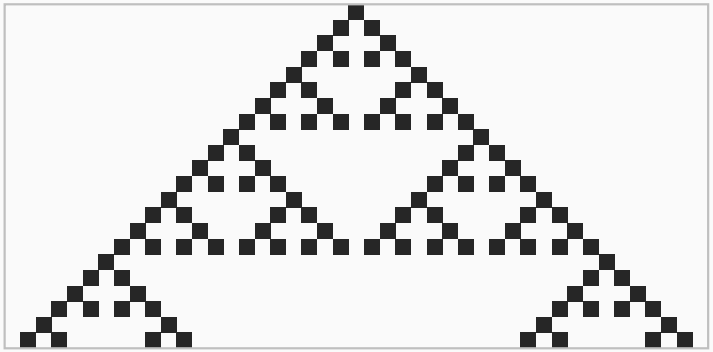
Rule 110: From a Single Black Cell



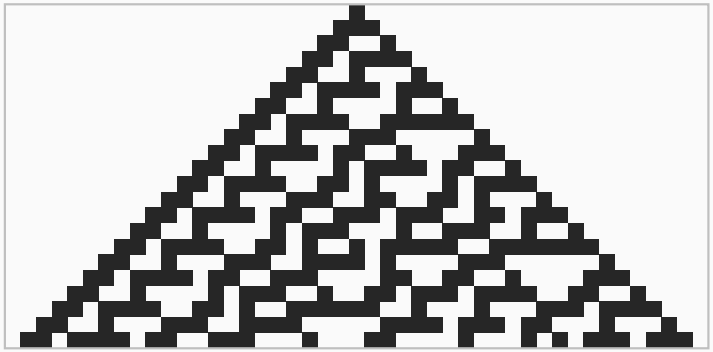
Each row is one time step. A trivial start, a tiny local rule — yet a complex pattern.

Complexity out of Simplicity

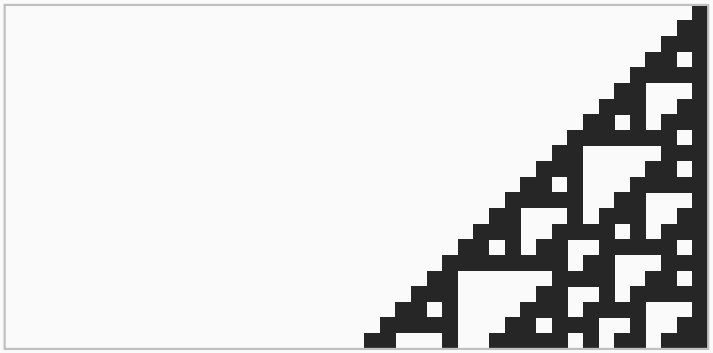
Rule 90 — fractal



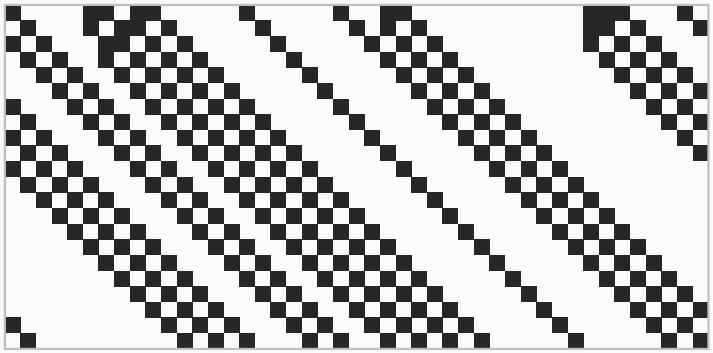
Rule 30 — randomness



Rule 110 — Turing-complete



Rule 184 — traffic flow



Each picture: one 8-bit rule, applied over and over.

Rule 110: How Does It ”Compute”?

To simulate a Turing machine M on input x:

1. **Encoding.** Translate M’s program and input x into a specific (long, structured) initial row of cells.
2. **Running.** Apply the Rule 110 table, one time step at a time.
3. **Halting.** M halts on $x \iff$ a specific pattern appears in the cellular automaton’s evolution.
4. **Reading off the result.** M’s output is recovered by decoding the region of cells around where the halting pattern appeared.

Alternative Model 3: Boolean Circuits

Basis. Fix a small set of gate types — here, $\{\mathit{AND}, \mathit{OR}, \mathit{NOT}\}$, with AND, OR taking 2 inputs and NOT taking 1 input:

$$\mathit{AND}(a, b) = a \wedge b \qquad \mathit{OR}(a, b) = a \vee b \qquad \mathit{NOT}(a) = \neg a$$

Circuit. A directed acyclic graph: input wires at the bottom carrying $x_1, \ldots, x_n \in \{0, 1\}$, gates from the basis wired together in layers, and one designated output wire. Values propagate straight through.

Claim. *Any* Boolean function $f : \{0, 1\}^n \to \{0, 1\}$ can be computed by some circuit.

Circuit Complexity

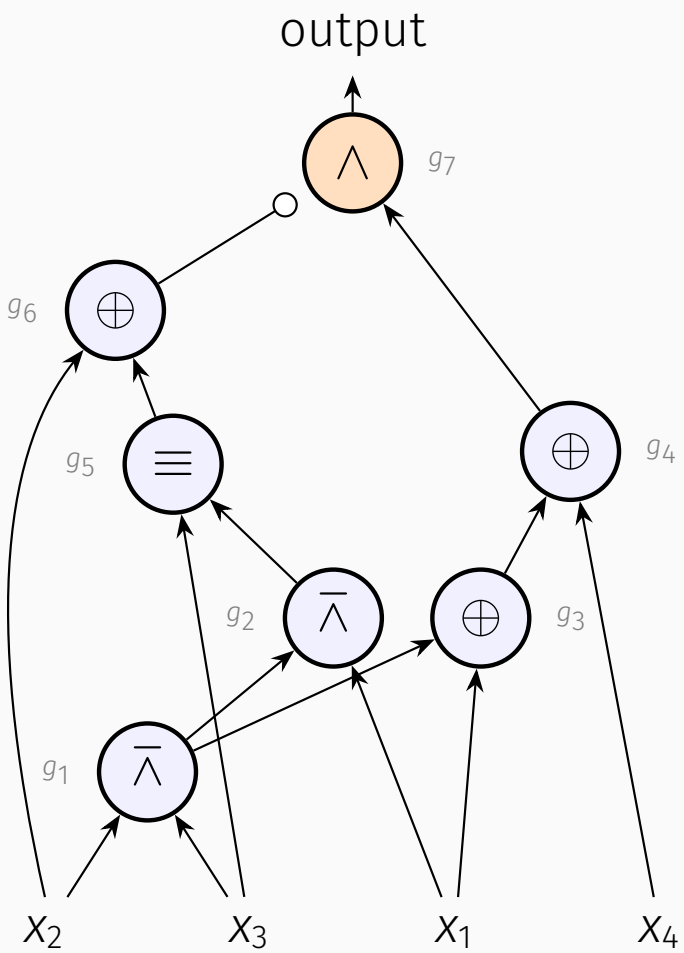
Definition 1
For a basis Ω, let $C_\Omega(f)$ be the minimum number of gates of a circuit over Ω computing f.

Basis	Gates
$\{\wedge, \vee, \neg\}$	AND, OR, NOT
B_2	all 16 functions $\{0,1\}^2 \to \{0,1\}$
$\{\overline{\wedge}\}$	NAND only

Lemma. For any two finite complete bases Ω, Ω', there is a constant $c = c(\Omega, \Omega')$ such that for all f,

$$C_\Omega(f) \leq c \cdot C_{\Omega'}(f).$$

An Optimal Circuit: MOD_3 on 4 Bits



$\mathrm{MOD}_3(x) = 1 \iff \sum x_i \equiv 0 \pmod 3$

$a\ b$	00	01	10	11
$\overline{\wedge}$ NAND	1	1	1	0
$\oplus$ XOR	0	1	1	0
$\equiv$ XNOR	1	0	0	1

$\circ$ on a wire $=$ negation, so $g_7 = g_4 \wedge \neg g_6$.

7 gates, and **no circuit with 6 gates exists** (SAT solver; Kojevnikov–Kulikov–Yaroslavtsev 2009).

General Circuit Lower Bounds

Holy Grail: prove a super-polynomial circuit lower bound for any problem in NP.

Best known, basis $\{\mathbf{AND}_2, \mathbf{OR}_2, \mathbf{NOT}\}$:

$3n - O(1)$	1984	Blum
$4.5n - o(n)$	2001	Lachish–Raz
$\mathbf{5n - o(n)}$	2002	Iwama–Morizumi

Full binary basis (all fan-in-2 gates):

$3n - o(n)$	1984	Blum
$\approx 3.01n - o(n)$	2016	Find–Golovnev–Hirsch–Kulikov
$\mathbf{3.1n - o(n)}$	2022	Li–Yang (Tsinghua)